Using a Fine-Tuned ResNet-50 Model

Bahar Nazeri -Qazal Moosavi

Abstract

Facial expression recognition is an important computer-vision problem with applications in human–computer interaction, behavioral analytics, and assistive technologies. This project fine-tuned a ResNet-50 backbone (pretrained on ImageNet) to classify face images into seven emotion categories: *angry, disgust, fear, happy, neutral, sad,* and *surprise*. The dataset was obtained from Kaggle (link above) and split into training, validation and test sets. Evaluation used cross-entropy loss, accuracy and F1-score. The experimental run produced near-perfect results on the training set (Accuracy $\approx$ 99%, F1 $\approx$ 99%) but moderate generalization on unseen data (Validation Accuracy $\approx$ 71%, Test Accuracy $\approx$ 69%). The large train $\rightarrow$ validation gap indicates severe overfitting and an implementation issue in the training loop that likely contributed to the behavior. This document reports methodology, results, six diagnostic figures (placeholders included), error analysis, prioritized recommendations, and ethical considerations.

Contents

1 Introduction **3**

2 Dataset (source and splits) **3**

3 Preprocessing and augmentations (high-level) **4**

4 Model and training setup (high-level) **4**

4.1 Critical implementation observation 4

5 Quantitative results **5**

6 Diagnostic figures (placeholders and ready analyses) **5**

7 Error analysis and recommended diagnostics **10**

8 Prioritized recommendations **11**

9 Ethical considerations and limitations **11**

10 Appendix: reproducibility and artifacts **12**

11 Conclusion **12**

1 Introduction

Facial expressions convey emotional states and intentions and are widely used in interpersonal communication. Automated recognition of expressions is therefore useful across many domains: affective computing, user-experience research, social robotics, mental-health screening, and video analytics. Major challenges include variability due to pose, illumination, occlusion, cultural differences, and the subtlety of some expressions.

Recent progress in deep learning—particularly convolutional neural networks and transfer learning from large-scale image datasets—has made it practical to build accurate image classifiers even when labeled domain data are relatively limited. In this project, a ResNet-50 model pretrained on ImageNet was adapted to the seven-class emotion recognition problem using the Face Expression Recognition Dataset from Kaggle. The goal was to train, evaluate, and analyze a robust pipeline and to identify concrete next steps to improve generalization and fairness.

This report is organized as follows: Section 2 describes the data and splits, Section 3 summarizes preprocessing and augmentations, Section 4 outlines the model and training setup at a high level, Section 5 reports quantitative results, Section 6 provides six diagnostic figures (placeholders) with ready-to-paste analyses, Section 7 prescribes error analysis and recommended experiments, Section 8 gives prioritized actionable recommendations, Section 9 covers ethics and limitations, and Section 10 lists reproducibility notes and appendix material.

2 Dataset (source and splits)

Source: Face Expression Recognition Dataset — Kaggle.
https://www.kaggle.com/datasets/jonathanoheix/face-expression-recognition-dataset

Classes: `angry, disgust, fear, happy, neutral, sad, surprise` (7 classes)

Data split used in the experiment:

- **Train:** 28,821 images
- **Validation:** 3,533 images
- **Test:** 3,533 images

Notes: The provided validation folder was split into validation + test halves. It is important to ensure there are no duplicated or near-duplicate images across splits to avoid leakage. Class imbalance is present; a weighted loss was used in the experiment to partially compensate. Per-class metrics are required for a complete assessment.

3 Preprocessing and augmentations (high-level)

- Images were resized to 224×224 pixels (model input).
- Training augmentations included random horizontal flip and small rotation (±10°).
- Normalization applied using ImageNet mean and standard deviation.
- Labels were encoded to seven integer classes.

These choices align inputs with the pretrained backbone. Stronger or class-aware augmentations (e.g., mixup, cutmix, color jitter, random erasing) and face detection + alignment may substantially improve generalization.

4 Model and training setup (high-level)

- **Backbone:** ResNet-50 pretrained on ImageNet; final head replaced for 7 outputs.
- **Trainable parameters (approx.):** 23.5 million.
- **Loss:** Weighted cross-entropy (class weights from per-class frequencies).
- **Optimizer:** Adam (learning rate = 1e-4).
- **Metrics:** Accuracy and multiclass F1.
- **Batch size:** 128.
- **Epochs executed:** 25.
- **Checkpoints:** Best-validation checkpoint saved (artifact noted).

4.1 Critical implementation observation

A training-loop issue was identified: gradients were zeroed *after* the optimizer step instead of *before* computing the forward/backward for the current batch. This produces unintended gradient accumulation (unless accumulation is intended) and can result in extremely low training loss and poor validation generalization. Fixing this is the top priority before further experiments.

5 Quantitative results

Table 1: Summary of final reported metrics

Phase	CrossEntropy Loss	Accuracy	F1-score
Train	0.02	0.99	0.99
Validation	1.50	0.71	0.71
Test	1.70	0.69	0.69

Interpretation: Extremely low training loss with near-perfect training metrics versus moderate validation/test performance suggests memorization/overfitting. The gradient-management bug likely contributed; class imbalance and limited augmentation are additional factors.

6 Diagnostic figures (placeholders and ready analyses)

Figure 1 — Training set class distribution

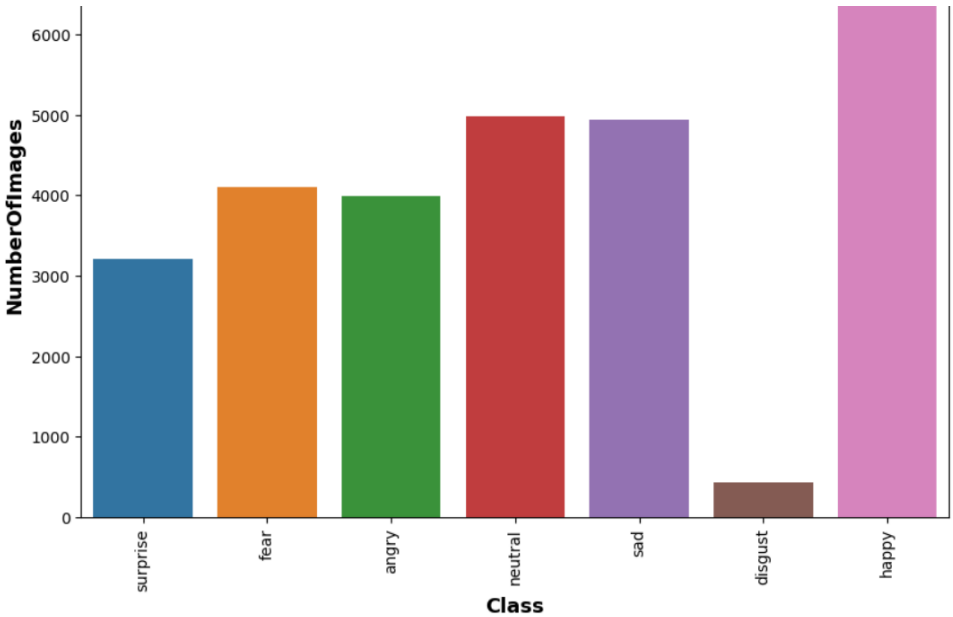

Figure 1: Training set class distribution.

Analysis: Figure 1 displays the number of training images per emotion class. The imbalance between classes is visually apparent: some labels have substantially more examples than others. Class imbalance tends to bias the classifier toward majority labels and reduces recall for minority classes. Although class-weighted loss was used to partially offset this imbalance, additional strategies such as oversampling minority classes, targeted augmentation, or focal loss could further improve per-class performance.

Figure 2 — Validation/Test set class distribution

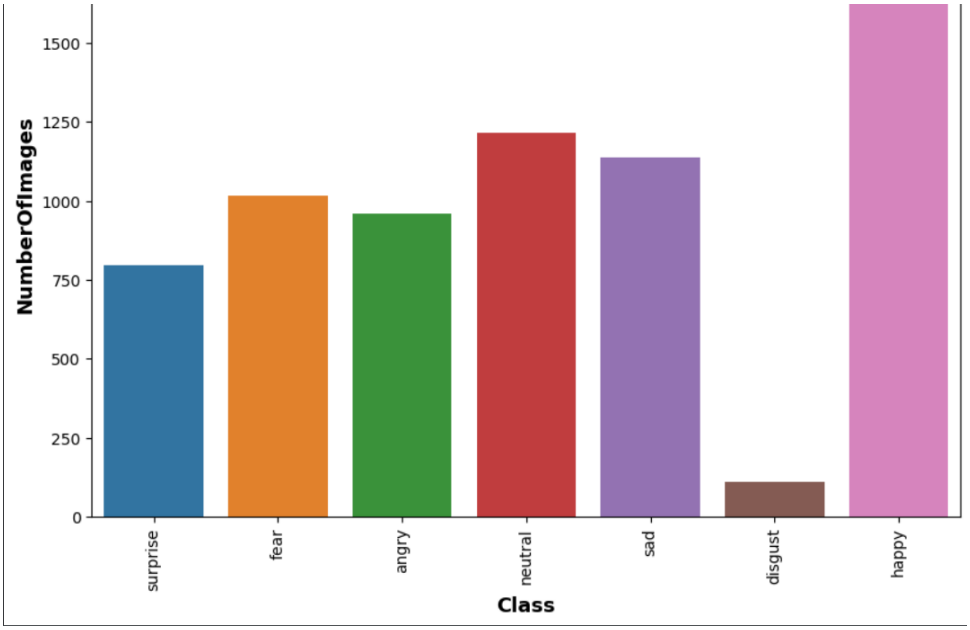

Figure 2: Validation/Test set class distribution.

Analysis: Figure 2 compares the validation/test distribution with the training distribution. Good practice is to have validation reflect the training distribution; any notable mismatch can explain degraded validation performance. If the validation distribution differs substantially, consider stratified sampling or adjusting evaluation metrics to reflect the intended deployment distribution.

Figure 3 — Representative training samples (image grid)

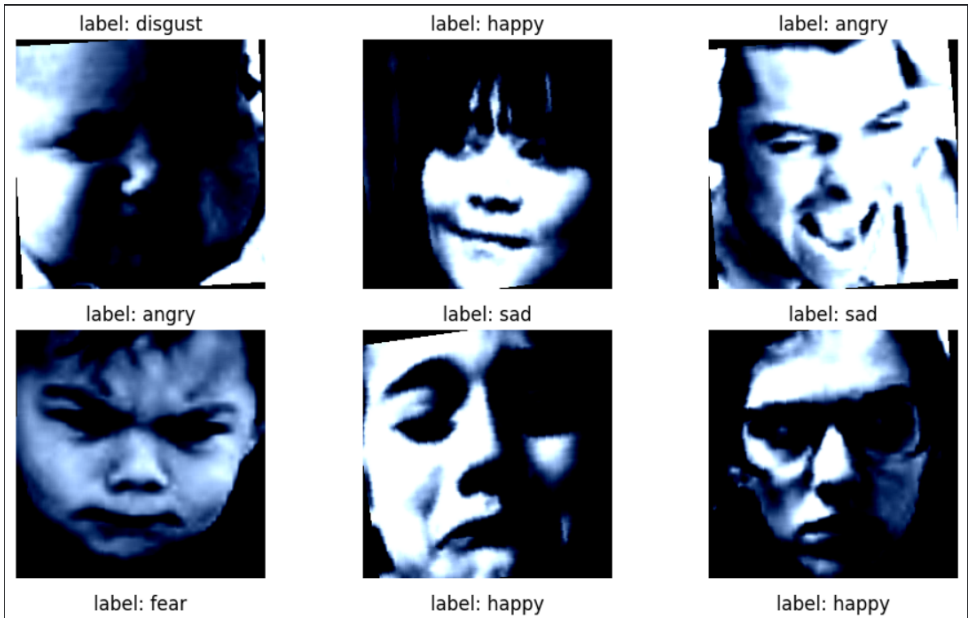

Figure 3: Representative training samples (3×3 grid).

Analysis: Figure 3 presents representative training images across different classes. Visual inspection reveals variability in pose, lighting, occlusion, and face framing. If faces are not consistently centered or crops include much background, adopting a face detection + alignment preprocessing pipeline will likely improve signal-to-noise ratio for the classifier. Also inspect whether some classes contain systematic differences (e.g., lighting, camera angle) that can bias the model.

Figure 4 — Loss curves per epoch (Train vs Validation)

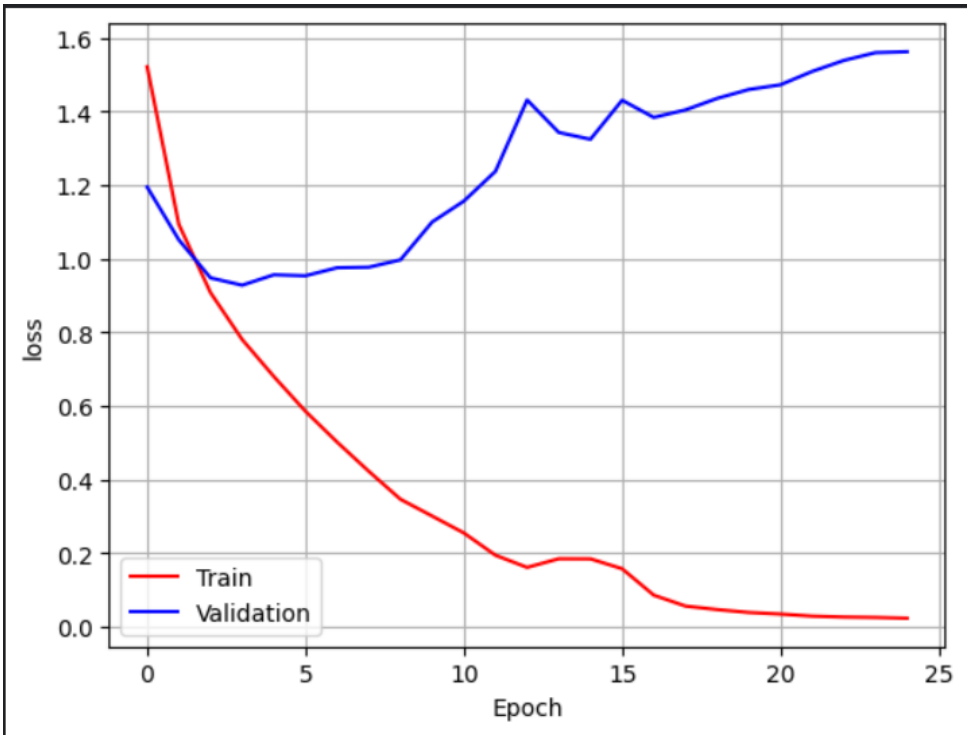

Figure 4: Training and validation loss vs epoch.

Analysis: Figure 4 shows a rapid decline in training loss to near zero while validation loss plateaus or increases after an initial decrease — a classic overfitting signature. Contributing factors include the gradient accumulation bug, lack of sufficient regularization (dropout, weight decay), and possibly limited augmentation. Recommended mitigations include fixing the bug, adding early stopping (on validation F1), enabling a learning-rate scheduler, and introducing stronger augmentation.

Figure 5 — Accuracy per epoch (Train vs Validation)

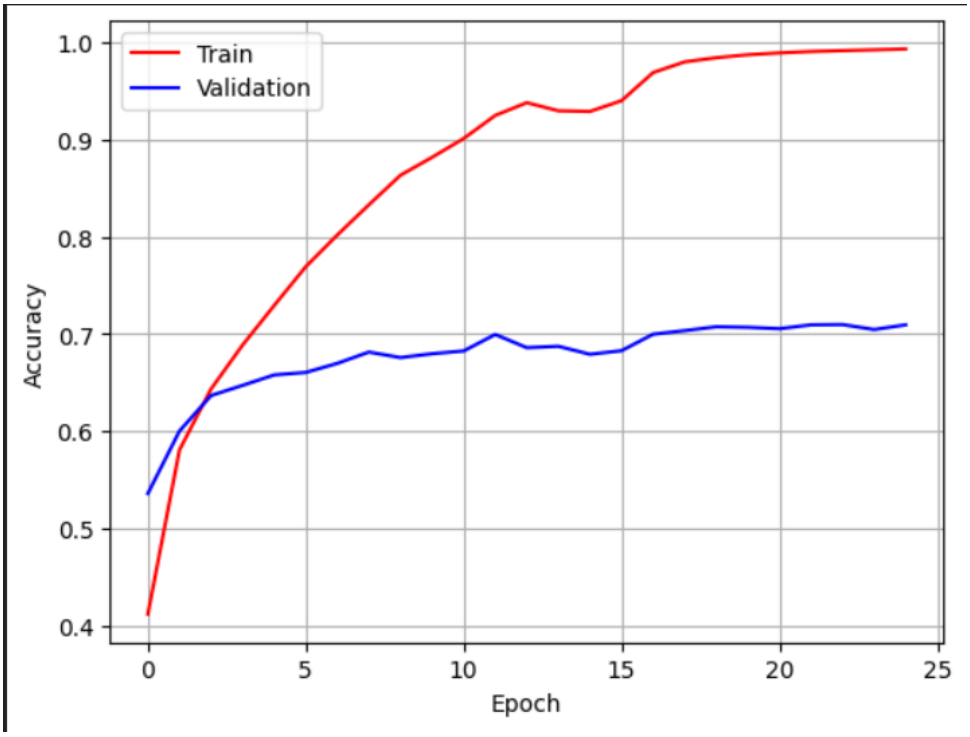

Figure 5: Training and validation accuracy vs epoch.

Analysis: Figure 5 illustrates training accuracy approaching 99% while validation accuracy remains around 70%. This large gap confirms memorization of training data. Consider strategies such as freezing early backbone layers and gradually unfreezing for fine-tuning, adding regularization, and performing architecture/augmentation ablation studies to identify robust improvements.

Figure 6 — F1-score per epoch (Train vs Validation)

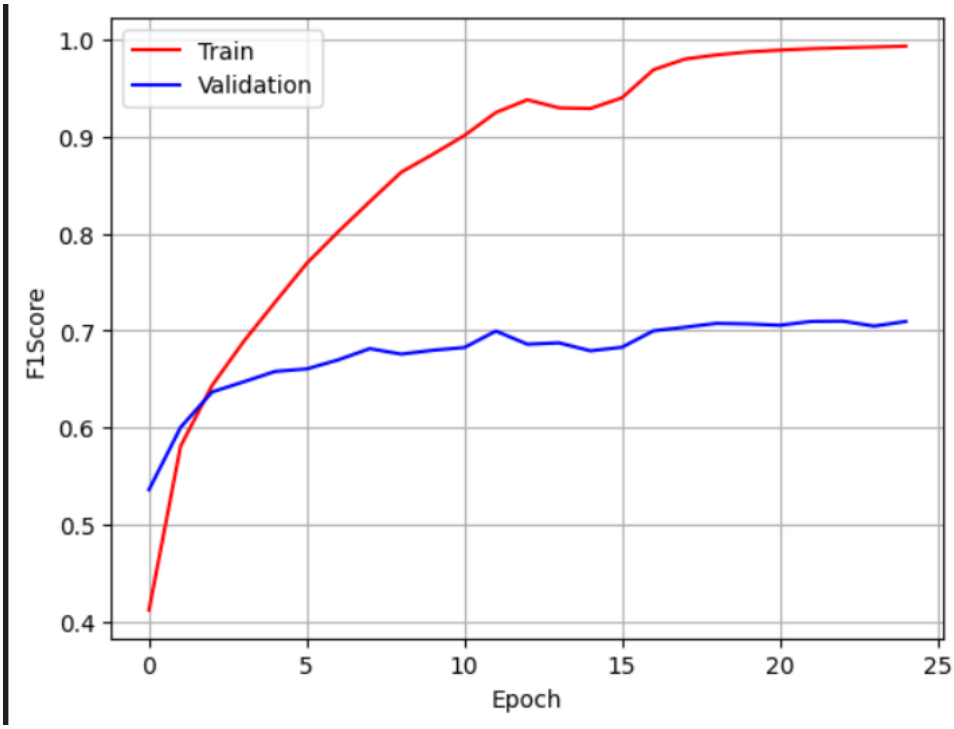

Figure 6: Training and validation F1-score vs epoch.

Analysis: Figure 6 (macro F1 or overall F1) mirrors accuracy trends and emphasizes the effect of class imbalance: macro F1 is sensitive to poor performance on minority classes. To diagnose, compute and display per-class F1 and a confusion matrix; this will reveal which classes are most frequently misclassified and guide targeted remedies.

7 Error analysis and recommended diagnostics

To understand and fix the model's shortcomings, perform the following diagnostics and include results in the report:

1. **Fix gradient-zeroing and retrain.** Reevaluate train/validation curves — if the gap reduces, the bug was causal.
2. **Confusion matrix (test):** compute per-class precision, recall, and F1; identify the most confused label pairs.
3. **Misclassified sample review:** inspect representative misclassified images with predicted and true labels.
4. **Activation/saliency inspection:** use Grad-CAM or other explanation methods to verify attention on face regions (eyes, mouth).

5. **Ablation experiments:** conduct controlled experiments to evaluate the effect of: (a) bug fix, (b) weight decay, (c) dropout, (d) augmentation strategies, (e) freeze/unfreeze schedules.

8 Prioritized recommendations

Immediate (must-do)

- Correct gradient handling: zero gradients *before* the forward/backward pass of each batch. Retrain and re-evaluate.
- Add early stopping based on validation F1 (patience = 3–7 epochs).

High-impact

- Add regularization: weight decay (L2) and dropout in the classification head.
- Use a learning-rate scheduler (e.g., ReduceLROnPlateau).
- Strengthen augmentations: color jitter, random crop, random erasing (cutout), mixup/cutmix.

Medium-impact

- Face detection and alignment preprocessing to standardize crops.
- Focal loss or targeted oversampling for minority classes.
- Freeze low-level backbone layers initially; fine-tune upper layers gradually.

Long-term / research

- Try alternative backbones (EfficientNet, ConvNeXt, ViT) or ensembles.
- Integrate uncertainty estimation (MC-dropout) for sensitive applications.
- Evaluate fairness across demographic subgroups and collect more representative data if necessary.

9 Ethical considerations and limitations

- **Decision risk:** Emotion-recognition errors can have serious consequences; do not deploy for high-stakes automated decisions without human oversight.
- **Bias and fairness:** Evaluate model performance across age, gender, and skin-tone groups; correct imbalances where found.

- **Privacy:** Facial data is sensitive. Ensure consent and compliance with relevant privacy laws.
- **Transparency:** Document limitations and expected error rates for users and stakeholders.

10 Appendix: reproducibility and artifacts

Environment (as reported)

- Python 3.11.13
- Key libraries: numpy, pandas, seaborn, scikit-learn, torch (CUDA-capable build in the experiment), torchvision, torchmetrics, OpenCV, PIL.

Saved artifacts

- Best validation model checkpoint (artifact noted during the run).
- Results CSV summarizing losses and metrics per phase.

Suggested assets to attach to submission

- Six diagnostic figures (place images into the placeholders above).
- Confusion matrix visual and per-class metrics table.
- Table summarizing ablation experiments (after bug fix).
- Hashes and environment spec for reproducibility.

11 Conclusion

This project implemented a ResNet-50 based facial expression classifier on the Kaggle Face Expression Recognition dataset. The experimental run showed excellent training metrics but only moderate generalization ($\sim$70% validation/test accuracy). A critical implementation bug in gradient management very likely caused or worsened the problem; fixing this is the immediate priority. After correcting the training loop, follow the prioritized recommendations (regularization, stronger augmentation, targeted sampling, and interpretability diagnostics) to improve generalization. Ethical considerations—bias, privacy, and transparency—must be addressed prior to real-world deployment.

References

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[2] Kaggle Dataset: Facial Expression Recognition Dataset. (Replace with full dataset citation or URL in final version.)

[3] PyTorch Documentation – `torchvision.models.resnet50`. (Replace with exact version/date if required.)